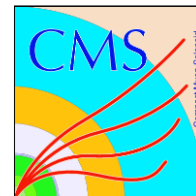


# Gridpack Generation

Brent Yates

[brent.yates@cern.ch](mailto:brent.yates@cern.ch)



THE OHIO STATE UNIVERSITY

# The basics

A gridpack contains the basic building blocks to instruct CMSSW (or other packages) on how to generate events

Get started here if you don't have access

We produce gridpacks on [CMS Connect](#) using a custom set of wrapper scripts which build on the official CMS submission tools

[https://github.com/TopEFT/GridpackGeneration/tree/SMEFTsim\\_run3](https://github.com/TopEFT/GridpackGeneration/tree/SMEFTsim_run3) (our scripts)

[https://github.com/TopEFT/GridpackGeneration/blob/SMEFTsim\\_run3/multilepton.md](https://github.com/TopEFT/GridpackGeneration/blob/SMEFTsim_run3/multilepton.md)  
(instructions for the multilepton Run 3 analysis)

# Anatomy of the repo

 SMEFTsim\_run3 ▾[GridpackGeneration](#) / [mcgeneration](#) /

- The magic happens in `configure_gridpack.py`
- The dir `scanfiles` contains `jsons` with our tested starting points and `txt` files with some of the random reweight points
- The dir `patches` contains various fixes, including for Run 3 the SMEFTsim patch to add an extra factor of  $g_s$  for WCs like  $c_{tG}$
- The `addons` dir contains the various models, process cards, and run cards
  - The process cards are in `<proc_name>_cards` (instructions for MadGraph on what final states we want)
  - The run cards are in the `templates` folder under `<EFT-proc_name>_template` (instructions for MadGraph on what kinematic settings we want)
- The dir `PLUGIN/CMS_CLUSTER` contains instructions on how to interface MadGraph with CMS Connect's condor service
- The dir `helpers` contains scripts to generate the necessary files to submit

# Scan files

The dir scanfiles contains `jsons` with our tested starting points and `txt` files with some of the random reweight points

- We find a starting point for each WC that scales the SM by  $\sim 10\%$
- We need  $\frac{(N+1)^2 - (N+1)}{2} = (N + 1)$  reweight points to reproduce the WC quadratic curve
  - We include another 10% incase a few points fail
- The script `make_scan_files.py` reads the `json` file and produces the `txt` files (if needed)
- Only run this if you need to update the txt files (e.g., a new starting point)

# The config script

- `configure_gridpack.py` is the master file
- We define the various processes using the `MGProcess` helper

```
78 # Processes used in TOP-19-001
79 ttHJet          = MGProcess(name='ttHJet'          ,process='ttH'  ,pcard='ttHJet.dat'      ,tdir='ttHJet_template')
80 ttlnuJet        = MGProcess(name='ttlnuJet'        ,process='ttlnu',pcard='ttlnuJet.dat'  ,tdir='ttlnuJet_template')
81 ttllNuNuJetNoHiggs = MGProcess(name='ttllNuNuJetNoHiggs' ,process='ttll' ,pcard='ttllNuNuJetNoHiggs.dat' ,tdir='ttllJet_template')
82 tHq4f           = MGProcess(name='tHq4f'           ,process='tHq'  ,pcard='tHq4f.dat'      ,tdir='tllq-4f_template')
83 tllq4fNoSchanWNoHiggs0p = MGProcess(name='tllq4fNoSchanWNoHiggs0p',process='tllq',pcard='tllq4fNoSchanWNoHiggs0p.dat',tdir='tllq-4f_template')
```

# The config script

- Next we define all the WCs and any relations they might have using the DegreeOfFreedom helper

- And further down the list of all WCs and processes we actually want to run over

```

102      ctGre      = DegreeOfFreedom(name='ctGre',relations=[[ 'ctGre' ] ,1.0])
103      ctGIm      = DegreeOfFreedom(name='ctGIm',relations=[[ 'ctGIm' ] ,1.0])
104      cQe        = DegreeOfFreedom(name='cQe' ,relations=[[ 'cQe' ],1.0])
105      cQe1       = DegreeOfFreedom(name='cQe1' ,relations=[[ 'cQe1' ],1.0])
106      cQe2       = DegreeOfFreedom(name='cQe2' ,relations=[[ 'cQe2' ],1.0])
107      cQe3       = DegreeOfFreedom(name='cQe3' ,relations=[[ 'cQe3' ],1.0])
108      ct1        = DegreeOfFreedom(name='ct1' ,relations=[[ 'ct1' ],1.0])
548  proc_list = [ttHJet,ttlNuJet,ttlNuNuJetNoHiggs,tHq4f,tllq4fNoSchanWNoHiggs0p,tttt,ttgamma]
549  dof_list  = [
550      cTHRe,cHQ1,ctWRe,ctBRe,ctGre,cbWRe,cHQ3,CHtbRe,CHt, # TOP-19-001 2-heavy
551      cQl31,cQl32,cQl33,cQl11,cQl12,cQl12,cQe1,cQe2,cQe3,ctl1,ctl2,ctl2,cte1,cte2,cte3, # TOP-19-001 4f
552      cleQt3Re11, cleQt3Re22, cleQt3Re33, cleQt1Re11, cleQt1Re22, cleQt1Re33, # lepton flavor split
553      cQj31,cQj38,cQj11,ctj1,cQj18,ctj8,          # 2-light 2-heavy quarks
554      #ctt,cQQ1,cQt1,cQt8,                        # 4-heavy quarks (Uncomment for tttt)
555      # New from SMP-24-003
556      clj1, CHbox, ctu1, ctb8, clu, cld, cQb8, ctd8, cQd1, cQd8, ctd1, cQu1, cbBRe, ctu8, cQu8
557  ]

```

# The config script

- Next we define all the WCs and any relations they might have using the DegreeOfFreedom helper

- And further down the list of all WCs and processes we actually want to run over

```

102      ctGre      = DegreeOfFreedom(name='ctGre',relations=[['ctGre'],1.0])
103      ctGIm      = DegreeOfFreedom(name='ctGIm',relations=[['ctGIm'],1.0])
104      cQe        = DegreeOfFreedom(name='cQe',relations=[['cQe'],1.0])
105      cQe1       = DegreeOfFreedom(name='cQe1',relations=[['cQe1'],1.0])
106      cQe2       = DegreeOfFreedom(name='cQe2',relations=[['cQe2'],1.0])
107      cQe3       = DegreeOfFreedom(name='cQe3',relations=[['cQe3'],1.0])
108      cTt        = DegreeOfFreedom(name='cTt',relations=[['cTt'],1.0])
548  proc_list = [ttHJet,ttlNuJet,ttlNuNuJetNoHiggs,tHq4f,tllq4fNoSchanWNoHiggs0p,tttt,ttgamma]
549  dof_list = [
550      cTHRe,cHQ1,ctWRe,ctBRe,ctGre,c
551      cQl31,cQl32,cQl33,cQl11,cQl12,
552      cleQt3Re11,cleQt3Re22,cleQt3
553      cQj31,cQj38,cQj11,ctj1,cQj18,ctj8, # 2-light 2-heavy quarks
554      #ctt,cQQ1,cQt1,cQt8, # 4-heavy quarks (Uncomment for tttt)
555      # New from SMP-24-003
556      clj1, cHbox, ctu1, ctb8, clu, cld, cQb8, ctd8, cQd1, cQd8, ctd1, cQu1, cbBRe, ctu8, cQu8
557  ]

```

NOTE: this list shows everything including  $t\bar{t}t\bar{t}$  but we usually submit that separately and uncomment the 4-heavy quark WCs for  $t\bar{t}t\bar{t}$  only

# The config script

- Then we set the model
- The process cards default to `SMEFTsim_topU3l_MwScheme_UFO` but we use the coupling string option to replace this with `SMEFTsim_top_MwScheme_UFO` and the proper settings for this model

```
569     gridpack = Gridpack(stype=stype,btype=btype,default_limits=[-20.0,20.0],restrict=restrict)
570     gridpack.setOptions(runcard_ops=rc_ops)
571     # For using a different model
572     gridpack.setOptions(coupling_string="SMHLOOP=0 NP=1 NPprop=0",replace_model=["SMEFTsim_topU3l_MwScheme_UFO","SMEFTsim_top_MwScheme_UFO"])
```

# The config script

- In main we have various settings like the tag (gets appended to all outputs), the scan type (FROMFILE means it will load the reweight points from a file), and the batch type (CMSCONNECT uses the condor batch system)

```
456  ✓ def main():
457      random.seed()
458      stype = ScanType.FROMFILE
459      btype = BatchType.CMSCONNECT
460      tag   = 'Run3_52Wcs_SMEFTsim_top'
461      restrict = False
462      runs   = 1                # if set to 0, will only make a single gridpack
463      npts   = 0
```

# Caveats

- The config script will skip any gridpacks already being processed
  - If you need to resubmit a gridpack either change the tag in the config script or delete all the files/dirs it made in the base area (`bin/MadGraph5_aMCatNLO/`) for that process
- The config script can sometimes take a few minutes to finish; it waits for the jobs to be picked up by condor
  - Either run with a stable connection, inside tmux from a reliable server, or run it as a background process with `python configure_gridpack.py &` (or even `nohup` if you need to log out)
- Some processes (like `t $\bar{t}$ γ`) can run for days in the codegen step
- The integrate step is a bit faster since it breaks things up in condor
- The reweight step can take a long time since it runs locally (maybe a day depending on the number of points)
  - You won't see anything on condor, but `ps -aux | grep $USER | grep grid` will show you what's still running